Q5

a. Verilog testbenches should be written so that they can be synthesized along with the DUT.
    i. False - Verilog testbenches are not synthesizable - they are only used to provide inputs for a module and can’t be loaded onto an FPGA.
b. Always blocks are only used for sequential logic design.
    i. False - I think that if all values are listed in the sensitivity list, and there are no points where the output is dependent on memory, an always block can technically be combinational? https://chipverify.com/verilog/verilog-always-block
c. When writing Verilog code, it is important to know about the hardware you are targeting to get the best synthesis result.
    i. True
d. FPGAs that are larger will always require more power but will allow you to have more complex designs.
    i. False - I think the power used also depends on how much of the FPGA is actually used? Larger FPGAs will let you run more complicated designs though, that bit is correct.
e. Non-blocking statements are better for modeling hardware. If you are going to synthesize a design you shouldn’t use blocking statements.
    i. False; blocking and non-blocking assignments are used for different things. The important thing here is to not use one when you mean the other!
f. Fix the following 5 Verilog code segments. These changes could be syntactic or functional. In the event you see multiple errors, choose the most functionally relevant (or mention all).

F1:

```
//Do not want an accidental synthesis of a latch
always@(posedge clk)
        casez (op)
                2’b1? : // assignment block 1
                2’b?1: // assignment block 2
        endcase
```

Error: no ‘end’ shown in always block.
Warning: 00 has no case and so data will remain unchanged, and 11 will only run the first case; not wrong, but it is annoying!

F2:

```
wire A, B, C
always @ (posedge clk)
begin
        A=C;
```

```
        B=A;
        C=B;
end
```

Error 1: this code, when run, has three competing lines of code that can get completed in different orders each time, changing the output of the block. Would introduce temporary memory or use blocking instructions.

F3:

```
Module foo (s, A, B, C, Z);
        input A, B, C;
        output Z;
        input [1:0] s;
        always @ (s or A or B or C)
                case (s)
                        3’b100 : Z<= A;
                        3’b010 : Z<= B;
                        3’b001 : Z<= C;
                endcase
endmodule
```

Error 1: the input s is 2 digits wide; the values that the case statement is looking for are 3 digits long. Maybe the person intended to use the values 3, 2, 1 instead of the values 4, 2, 1?

F4:

```
Module adder_4_bit (A, B, C_in, Sum, C_out);
        Input [3:0] A, B;
        Input C_in;
        Output [3:0] Sum;
        Output C_out;
        Wire [3:0] A, B, C_in, Sum, C_out;

        fulladder FA3(A[3], B[3], C2, Sum[3], C_out);
        fulladder FA2(A[1], B[2], C_in, Sum[2], C2);
        fulladder FA1(A[2], B[1], C0, Sum[1], C1);
        fulladder FA0(A[0], B[0], C1, Sum[0], C0);
endmodule
```

Error 1: I believe that for fulladder FA2 and FA0, the C_in inputs are switched; they should read:

```
        fulladder FA0(A[0], B[0], C_in, Sum[0], C0);
        fulladder FA2(A[1], B[2], C1, Sum[2], C2);
```

F5:

```
//Should be a truly combinational circuit
always @ (x or y)
begin
w = y & z;
u = w | x;
end
```

Error 1: ‘z’ is missing from the always sensitivity list, assuming that this is the only place where w and u are changed. w is updated by y and z, u is updated by w and x, but z is not on the sensitivity list.

g. There is a group that is planning on creating a driving simulation game for their project.
    i. False - I don’t believe anyone is planning on making a game at all. The projects as far as I’m aware are:
    FPGA Qubit Controller
    Ray Tracing Core
    Secure Computing Enclosure using Sequestered Encryption
    FPGA-Based Real-Time Vital-Signs Monitor
    Live Audio Frequency Visualizer
    FPGA-Based Finite Impulse Response (FIR) Filter
    Image processing pipeline
    HFT-FPGA System
    (referenced from https://piazza.com/class/mtd4fkguc682l9/post/16)
h. Using the “reg” syntax must lead to a register being synthesized.
    i. False: sometimes the register that is simulated will get synthesised out.
i. Circle the correct ordering of the digital design flow. (iii)
    i. Design spec -> simulation -> design entry -> design partition -> synthesize
    ii. Design spec -> design entry -> design partition -> simulation -> synthesize
    iii. Design spec -> design partition -> design entry -> simulation -> synthesize
    iv. Design partition -> design spec -> design entry -> simulation -> synthesize
j. Verilog uses “Event Driven Simulation”.
    i. True
k. How many functions can a 4-input LUT represent? (iv)
    i. 16
    ii. 4
    iii. Infinite
    iv. 65536 (16 possible output slots, each can either be 1 or 0, so 2^16)
l. We read two papers in class related to cyclic combinational logic circuits.
    i. What were each about? (2-3 sentence for each paper)
        1. The Cyclic to Acyclic paper described how to convert the combinatorial portions of a circuit (for inputs where the output doesn’t oscillate or require stored memory) into a separate acyclic circuit. This was done by

creating and then merging ‘acyclic fragments’. The author presents a proof of the statement “For a circuit with a strongly-connected component (SCC) to behave combinationally, at least one input to a gate in the SCC must be driven to a controlling value.”, and uses said proof to demonstrate examples of cyclic-to-acyclic conversion. They discuss their prototype to automate this, and mention future work.

2. The Acyclic to Cyclic paper describes the benefits of synthesising an apparently cyclic circuit, despite its generally more complicated behaviour, and makes clear the difference between topological feedback and electrical feedback in circuits and the importance of non-oscillation. The author goes through their algorithm to create small loops to minimise work done, presenting the “Build Up Approach” and the “Break-Down Approach”. They look at the cost of equivalent cyclic and acyclic circuits, where the cost metric is measured by the number of literals used. They end by calling for a paradigm shift in the field.

ii. Provide two questions you made for each paper for the paper game.

1. Cyclic to Acyclic: https://github.com/ddsva1/bu-EC551/blob/main/required-reading/making-cyclic-circuits-acyclic-questions.txt
    a. How does the merging algorithm avoid creating a cycle?
    b. How do we choose which acyclic circuit fragments to discard?
2. Acyclic to cyclic: https://github.com/ddsva1/bu-EC551/blob/main/required-reading/synthesis-of-cyclic-combinational-circuits.pdf
    a. What was Malik's work on?
    b. What is a substitutional set?

m. What are suggestions you have for improving the class?

i. Of course a bigger, closer-to-Engineering, better room than CGS 121 would be preferred! I can’t help but think that it would be easier for you to teach and for us to see the board in a different room.

ii. I would prefer smaller individual projects instead of doing the ‘individual’ courseworks in teams of 2 with the amount of work scaled up.

It was hard to find and organise a team for this, if you don’t come into the class with teams you are accustomed to from previous courses, which is leaving my team with not enough time to complete the coursework!

Ideally, I’d prefer the ability to enter a random pairing of names of people who don’t have a preferred team at the start of the project or smaller, genuinely individual projects.

iii. If you have covid recordings for the lectures, would it be possible to upload those to blackboard to provide another way to revise? If not, absolutely fine

iv. Great format for the exam!